LISA: LLM Informed Systemverilog Assertion generation with RAG and Chain-of-Thought

Subhajit Paul*, Ansuman Banerjee*, Sumana Ghosh*, Sudhakar Surendran†, and Raj Kumar Gajavelly‡
*Indian Statistical Institute, Kolkata, India, Email: {subhajit.paul_t, ansuman, sumana}@isical.ac.in
†Texas Instruments India, Email: sudhakars@ti.com, ‡IBM Systems India, Email: rgajavel@in.ibm.com

2025 IEEE Computer Society Annual Symposium on VLSI (ISVLSI) | 979-8-3315-3477-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISVLSI65124.2025.11130226

***Abstract*—Assertion generation for Formal Verification (FV) needs an in-depth understanding of assertion language constructs, design fundamentals and RTL implementation techniques. Assertions are derived from a design's natural language (NL) specification, but this is hindered by (i) NL ambiguity, leading to multiple interpretations, and (ii) the need to validate translations, as errors may occur. To address these issues, we propose LISA, an interactive framework leveraging large language models (LLMs), a formal verification tool, and human guidance. It systematically translates an NL specification into SystemVerilog Assertions (SVA) that accurately capture the intended behavior. LISA begins by breaking down complex NL descriptions into a structured verification plan using Chain-of-Thought (CoT). It then generates SVAs for each item in the verification plan, formally verifies them using an FV tool, and analyzes statement coverage. For coverage gaps, LISA refines the assertions in an iterative feedback loop to enhance quality and correctness with the help of Retrieval Augmented Generation (RAG). We conduct a comprehensive LLM-assisted NL to SVA generation case study, producing syntactically correct SVAs with high statement coverage for the OpenTitan IPs, achieving 100% syntactic correctness with zero or one-shot prompting.**

***Index Terms*—SystemVerilog Assertion, Assertion generation, Code Coverage, Formal Verification, Large Language Model** [1]

I. Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation in diverse tasks that traditionally require human expertise, including code generation [1], language understanding [2]. This has created new opportunities in FV, particularly in the automated generation and verification of system properties, where mathematical precision and rigorous proofs are of paramount importance. In the FV community today, System Verilog Assertions (SVAs) serve as the primary language for expressing design specifications and constraints. Their significance extends beyond mere specification; they facilitate bug detection in early development stages and enable comprehensive coverage analysis. However, crafting effective assertions demands a thorough understanding of SVA constructs and experience in protocol specification understanding, which often creates a bottleneck in the verification process.

Recent studies highlight the potential of LLMs in assertion mining and generation. Zhang et al. introduced AssertCraft [3], a fine-tuned LLM-based framework. Similarly, [4] demonstrated the efficacy of models like CodeT5 in unit test assertion generation, surpassing baselines in accuracy and bug detection. Hassan et. al. in their work [5], used GPT4 to generate invariants and formal models from system specifications and mutation tests. Similarly, works like [6], [7] demonstrated LLMs' ability to streamline assertion generation from natural language specifications and reduce manual effort. Previous approaches [8] [9] have generated syntactically correct assertions but failed to achieve adequate coverage or capture temporal dependencies crucial for hardware verification. Standard LLMs without specialized training often lack access to domain-specific verification patterns required for comprehension of complex protocols [10], making them prone to inaccuracies.

In this work, we address these challenges by integrating LLMs with verification tools to enhance assertion correctness and coverage. We introduce **L**LM **I**nformed **S**ystemverilog **A**ssertion generation (**LISA**), a novel framework that transforms natural language specifications into structured SVAs using a systematic, step-by-step reasoning approach using Chain-of-Thought (CoT) prompting [11] with one / two attempts (also called 1-shot / 2-shot in LLM parlance). We enable LLMs to generate syntactically correct assertions with high coverage, as demonstrated by comprehensive coverage reports from an industry-leading FV tool [12]. LISA employs an iterative refinement process and an in-built RAG to improve assertion quality and reliability. Our key contributions include:

- ***SVA Generation with Minimal Retries:*** Unlike prior methods that often require multiple iterations to produce syntactically valid assertions with good statement coverage, LISA achieves correct SVAs in 0 / 1 shot.
- ***CoT-based Prompt Design and RAG Integration for FV:*** We use Retrieval-Augmented Generation (RAG) and Chain-of-Thought (CoT) prompting. Our framework uniquely leverages the multimodal capabilities of LLMs to interpret diagrams, enhancing assertion accuracy.
- ***Coverage-Driven Refinement:*** Unlike existing approaches that focus only on correctness, LISA introduces a coverage-aware feedback loop. By analyzing uncovered RTL lines, retrieving relevant design insights using RAG, and refining assertions iteratively, we systematically improve coverage.
- ***Case Study:*** We evaluate LISA on OpenTitan [10] IP specifications, demonstrating its practical impact on hardware verification. We use the popular Claude 3.5 Sonnet [13] as

[1]This work is fully supported by the Semiconductor Research Corporation .

979-8-3315-3477-6/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on March 14,2026 at 05:07:33 UTC from IEEE Xplore. Restrictions apply.

the backbone and a FV tool for our experiments.

II. Background & Related Work

LLMs are neural network-based architectures trained on vast text corpora, capable of understanding and generating human-like text. Some important terminologies relevant to us are:

- *Chain-of-Thought (CoT)*: A prompting technique that enables LLMs to break down tasks requiring logical reasoning into intermediate reasoning steps, improving accuracy.
- *Retrieval-Augmented Generation (RAG)*: A technique combining embedding models with knowledge retrieval to increase accuracy and reduce hallucinations.
- *Multimodality*: Recent LLMs handle multiple modalities, including text, image, audio, and sensor data.

LLMs have demonstrated competence in generating syntactically correct code from natural language [14]. [11] showed CoT prompting enhances code generation but did not address hardware verification's temporal reasoning challenges. Information extraction transforms unstructured natural language into structured data, which is critical for formal verification, and tools such as [2], [15] have been used to do that job. [9] introduced the JSON-based decomposition of design specifications but lacked reproducibility due to unspecified prompt engineering techniques. [16] proposed extracting verification test plans from design contexts but required manual intervention to convert these into SVA. Recent works have extended LLM-assisted assertion generation to security-critical IPs. [17] explored security assertion generation using LLMs to detect vulnerabilities in SoC verification, while [6] investigated LLM applications in security assertion generation. [18] fine-tuned LLMs on Verilog datasets. [8] identified issues in LLM-generated SVAs, such as incorrect timing specifications, and addressed them using rule-based constraints. AutoVCoder [19] demonstrated the effectiveness of RAG and a knowledge retriever to improve domain-specific performance. While AssertCraft uses subtask-focused fine-tuning to generate assertions, its applicability is limited to simpler designs. ChIRAAG utilizes LLM for assertion generation from specifications but primarily supports simulation-based verification. AutoVCoder focuses on Verilog code synthesis, offering limited support for assertion generation. However, to the best of our knowledge, there is no work on integration with RAG and structured CoT that can help an LLM overcome hallucinations and produce better-quality assertions with a lesser number of retries.

Existing LLM-assisted assertion generation approaches primarily focus on using the textual modality of LLMs and syntactical correctness while neglecting coverage metrics, leading to incomplete verification and increased debugging, which leads to increased token cost. Our approach advances existing work by integrating the multimodal feature with CoT prompting and RAG for improved temporal reasoning in assertion generation. Unlike prior methods, we emphasize achieving higher coverage metrics while reducing the number of retries. Empirical evaluations indicate that our method generates correct assertions in 0 or 1 retry in most cases, a notable improvement over existing approaches.

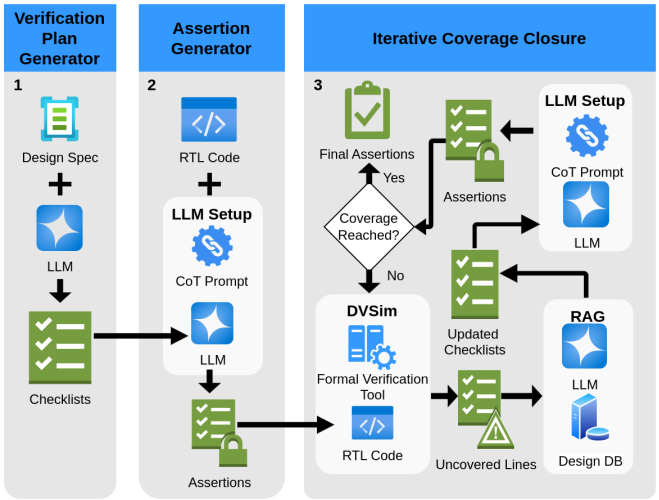

Fig. 1: 3-phase workflow of LISA

III. LISA Architecture

In this section, we propose LISA, an LLM-backed assertion generation framework that generates SVAs from NL specifications and helps to improve statement coverage using industry-standard FV tools and RAG. Figure 1 illustrates the major building blocks and interactions. LISA consists of:

- *Verification Plan Generator:* This module takes in protocol specifications in the form of text, images and produces a verification plan as a set of test items.
- *Assertion Generator:* This generates assertions with the CoT prompt and the verification plan produced above.
- *RAG-based Iterative Coverage Closure:* This module iteratively improves coverage of generated assertions using RAG.

In the subsequent sections, we discuss these blocks in detail.

A. Verification Plan Generator

Given an NL specification, using an LLM and a prompt, LISA systematically transforms informal NL requirements into structured verification tasks. The results are formatted as a tabular verification plan consisting of test scenarios that must be handled during assertion implementation. A verification plan is a collection of checks, where each check identifies a protocol scenario that the LLM extracts from the specification that is crucial for the verification process. One of our main contributions is utilizing multi-modal capabilities of LLMs to interpret diagrams in the protocol specification.

- **Input:** Protocol specifications containing both textual descriptions and diagrams (e.g. block diagrams, timing diagrams, component diagrams etc).
- **Processing:** LLMs analyze diagrams to extract explanations and these are merged with the NL specification.
- **Output:** A JSON document consisting of a table of checks. This table includes for each check a unique ID, check name, and description, providing traceability between specifications and verification. The list of checks is a set of verification scenarios that constitute the verification plan.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on March 14,2026 at 05:07:33 UTC from IEEE Xplore. Restrictions apply.

The multi-modal input specification is sent to the LLM with the prompt in Figure 2.

> Assume you are a formal verification engineer specializing in validating digital circuit functionality through formal property verification to confirm design implementation meets specifications. Review the provided specifications to understand the design. Then complete these tasks to develop a comprehensive formal verification test plan with checkers, constraints, and functional covers.
>
> - Create a summary of the specification in two parts: the design's core functionality and its operational flow.
> - Develop a thorough list of checkers to verify all design functions. Ensure checkers can be implemented using only the design's inputs and outputs.
> - Present the checkers, constraints, and functional covers in a 4-column table format. Include a unique ID for each checker (CHK1, CHK2, etc.), the checker name, and a detailed description. **Important**: Verify completeness and avoid any redundancy in the checkers.
> - Propose a staging plan for checker implementation and debugging.
> - Format the staging plan as a 3-column table with Stage, Objectives, and Tasks (including implementations for each stage).

Fig. 2: Our LLM prompt for Verification Plan generation

B. Assertion Generator

State-of-the-art assertion generation approaches either produce simplistic checks or require significant expert guidance [9]. To address these limitations, we introduce a structured prompt that enables the LLM to articulate the reasoning behind each assertion. This explicit reasoning process ensures assertions are functionally meaningful and aligned with design intent. In this step, we use the list of checks generated in the previous step, providing the Design Under Test (DUT) code as additional context. Inferring the notions of CoT and rulesets from [8], we introduce a structured prompt as follows.

- **Intermediate Thought Generation:** The prompt guides the LLM to generate intermediate reasoning steps before producing assertions from checklists. This ensures that assertions are syntactically correct and meaningful.
- **Assertion Planning:** Ensures the LLM understands and generates a structured list of planned assertions, including involved signals and sequences.
- **Rule Incorporation:** Uses a set of predefined rules from [8] to constrain the LLM's assertion generation process, reducing syntactical errors.
- **Final SVA Formatting:** Enforces the SVA format, ensuring the generated assertions adhere to proper syntax.

The RTL code of the DUT is also passed onto the LLM in this stage, with the prompt in Figure 3. On analyzing the generated assertions, we see that LLMs can comprehend and summarize multiple items of the verification checklist into SVAs.

> Assume the persona of a formal verification engineer, an expert in verifying the functionality of digital circuits using formal property verification to ensure that all the design implementation adheres to the design specification. Your primary task will be to implement assertions from the given Checker List for a particular design.
>
> - Please go through the RTL file and checklists provided as attachments and understand the overall functionality of the design. Then, execute the following steps to create a set of accurate assertions.
> - **Assertion Planning**:
> *Thought*: What type of assertion would best capture each identified behavior? You can choose between immediate and concurrent System Verilog Assertions.
> *Action*: Determine the assertion type (Immediate or Concurrent), relevant signals, and sequences for each assertion.
> *Output*: List planned assertions with their types, involved signals, and sequences
> - **Syntax Formulation**:
> *Thought*: How do I translate each planned assertion into proper SystemVerilog assertion syntax?
> *Action*: Construct each assertion, carefully considering temporal operators and signal and sequence relationships. Follow these guidelines strictly. **Ruleset from [8] also included but not shown here due to shortage of space.**
> *Output*: Write out each assertion using SVA syntax.
> - **Final SVA Generation**:
> *Thought*: How should I structure the final SVA block for maximum readability and effectiveness?
> *Action*: Compile the refined assertions into a complete, well-organized SVA block. The SVA file should strictly follow this format.
> - Make sure the assertions reside inside the same RTL file
> - Make sure the signal widths are correctly declared
> - Make sure the declared signal names match the design signal names in the checklist.
> - Add Glue Logic or Helper code to simplify the assertion expressions wherever possible.
>
> *Output*: Present the final SVA block, including any necessary module declarations or comments.

Fig. 3: Our LLM prompt for Assertion generation.

C. Iterative Coverage Closure

Coverage metrics are essential to measuring the quality of LLM-generated assertions. In this phase, we aim to improve the *statement coverage*, a metric that quantifies how thoroughly each line of the RTL code is exercised by the assertions. As depicted in Figure 1, we use DVSim, a utility tool inside OpenTitan [10] and a FV tool under the hood to formally verify the LLM-generated SVAs. The SVAs are included inside the RTL code of the DUT and passed to DVSim. The statement coverage model inside the FV tool tracks whether every statement in the code has been exercised during formal verification by the given set of assertions and produces a list of lines that are not covered. To understand the metrics, it is important to clarify different coverage concepts.

- **Formal Coverage (F Cov)** – This represents the overall verification effectiveness, combining both Stimuli and Checker coverage. For an item to be covered formally, it must be covered by both Stimuli and Checker coverage.
- **Stimuli Coverage (S Cov)** – Stimuli coverage represents the reachability of the formal environment. Covered items have been reached by formal traces.
- **Checker Coverage (C Cov)** – This assesses how well the generated assertions validate the exercised design behavior based on Cone of Influence (COI) coverage.

Given the list of SVAs generated in the previous steps, we do the following to improve coverage.

- We employ a RAG-based agent that provides *explanations* and *an augmented verification plan* using the LLM to cover.
- We extract explanations for uncovered lines using RAG, asking it to *explain these lines in the context of the DUT*.
- The framework from the explanations and checklists then generates a new set of SVA for the uncovered lines.
- For the newly generated assertions, we again invoke DVSim to check for coverage improvements.

A challenge in hardware verification is achieving comprehensive statement coverage, particularly for complex SystemVer-

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on March 14,2026 at 05:07:33 UTC from IEEE Xplore. Restrictions apply.

ilog code patterns, leading to coverage gaps. Our RAG system helps assertion generation in two key ways:

- incorporates relevant context from the entire codebase to better understand complex constructs, and
- automatically updates the verification plan for uncovered lines, systematically improving coverage.

We first clean the RTL design code by removing comments to reduce the token size in future prompts. We then break down the code into smaller, manageable segments of 500 tokens each, with a 50-token overlap to maintain context across segments. Segmentation ensures related code constructs stay together, which is crucial for understanding hardware-specific patterns. We use all-MiniLM-l6-v2 [20] that outputs 384-dimensional vectors to capture the semantic relationships of each code chunk. As depicted in Figure 1, we use the Facebook AI Similarity Search (FAISS) [21] tool to store the embedding vectors in the *Design DB* for similarity searches in our Question Answering agent.

Question-Answering Agent: Our question-answering agent retrieves relevant context from the design database to explain uncovered lines and improve assertion generation. As depicted in Figure 1, when an uncovered line is detected, the system first vectorizes it and performs a similarity search against the pre-indexed vector database (Design DB). This retrieval step ensures that semantically related code chunks are fetched as contextual input for the next stage. The retrieved context is fed into the LLM along with a structured CoT prompt. The LLM uses this information to generate explanations for uncovered lines, leading to an updated checklist. This iterative process refines the verification plan, ensuring that newly generated assertions target previously uncovered lines.

IV. Case Study

We present a case study on the Hardware Message Authentication Code (HMAC) IP from OpenTitan to evaluate our framework's effectiveness.

```
module hmac
...
  assign idle = !reg_fifo_wvalid && !fifo_rvalid
                && hmac_core_idle && sha_core_idle;

  prim_mubi_pkg::mubi4_t idle_q, idle_d;
  assign idle_d = prim_mubi_pkg::
      mubi4_bool_to_mubi(idle);
  always_ff @(posedge clk_i or negedge rst_ni)
      begin
    if (!rst_ni) begin
      idle_q <= prim_mubi_pkg::MuBi4False;
    end else begin
      idle_q <= idle_d; // (Uncovered Line)
    end
  end
  assign idle_o = idle_q;
...
endmodule
```

Listing 1: Syatem Verilog Code Snippet from the HMAC Module

From the entire HMAC RTL code, a representative snippet is shown in Listing 1, which determines when the HMAC module is idle, encodes the idle status using a multi-bit format for reliability, and registers it synchronously to ensure stability.

A. Verification Plan and Assertion Generation

We first consider the HMAC design specification and the SHA256 algorithm block diagram to generate a comprehensive verification plan as a list of check scenarios. Then this list of checks with this complete RTL file and our CoT prompt is passed onto the assertion generator. The assertion generator provides a set of SVAs for each checklist. During the verification plan and assertion generation, the framework produces reasons behind the checklists and assertions. Listing 2 demonstrates this structure, where the first comment represents the verification check, and the second comment provides the corresponding LLM-generated property description. In all, we have 15 checks and 13 corresponding assertions.

```
// | CHK1 | Mode_Selection | Verify that mode
    selection between SHA-2 and HMAC works
    correctly: When SHA-2 mode is selected, key
    processing should be bypassed; When HMAC mode
    is selected, two-round processing should be
    enabled |
// Property: When SHA-2 mode is selected (hmac_en
    =0), key processing should be bypassed
property p_sha2_mode_key_bypass;
  @(posedge clk_i) disable iff (!rst_ni)
    (!hmac_en && sha_en) |-> !hmac_fifo_wsel;
endproperty
assert_sha2_mode_key_bypass: assert property(
    p_sha2_mode_key_bypass);
```

Listing 2: An example verification check and its implemented assertion

Next, we include these SVAs in the HMAC module and check for syntactic correctness. The LLM has some inconsistent outputs related to property name re-declarations and misspelt signal names. We fix them and pass them onto the FV tool to find the uncovered code lines in the HMAC module. Listing 1 highlights one such uncovered line.

B. Iterative Coverage Closure and RAG

To address coverage gaps, we implement a RAG that:

- Chunks and embeds HMAC IP-related files in a FAISS vector database.
- Performs similarity searches for uncovered lines.
- Generates code line explanations and verification plan

Listing 3 shows additional assertions generated for the uncovered line in Listing 1, focusing on reset behavior, value updates, and stability conditions. Including these additional assertions in the HMAC module improves formal statement coverage (F Cov), increasing from 93.59% to 96.15% (↑2.56%). Similarly, the checker statement coverage (C Cov) rises from 93.6% to 96.15% (↑2.55%) while the stimuli statement coverage (S Cov) remains 100%, indicating no gaps in test stimulus execution across stages. Although these stages resolve the issue for HMAC, it does not consider the files included in the RTL. The generated assertions cannot cover a few lines of the files that are being included. We perform a similar operation on the other files to improve coverage.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on March 14,2026 at 05:07:33 UTC from IEEE Xplore. Restrictions apply.

```
// Property: Verify idle_q takes MuBi4False value
    immediately after reset
property idle_q_reset_value_p;
    @(posedge clk_i) disable iff (0)
    (!rst_ni) |-> (idle_q == prim_mubi_pkg::
        MuBi4False);
endproperty
IDLE_Q_RESET_VALUE: assert property(
    idle_q_reset_value_p);
// Property: Verify idle_q captures idle_d value
    in the next clock cycle when not in reset
property idle_q_update_p;
    @(posedge clk_i) disable iff (!rst_ni)
    (1'b1) |=> (idle_q == $past(idle_d));
endproperty
IDLE_Q_UPDATE: assert property(idle_q_update_p);
// Property: Verify idle_q maintains its value
    when idle_d is stable
property idle_q_stable_value_p;
    @(posedge clk_i) disable iff (!rst_ni)
    ($stable(idle_d)) |=> ($stable(idle_q));
endproperty
IDLE_Q_STABLE_VALUE: assert property(
    idle_q_stable_value_p);
// Property: Verify idle_q is not affected by
    idle_d changes during reset
property idle_q_reset_immunity_p;
    @(posedge clk_i) disable iff (0)
    (!rst_ni && $changed(idle_d)) |-> (idle_q ==
        prim_mubi_pkg::MuBi4False);
endproperty
IDLE_Q_RESET_IMMUNITY: assert property(
    idle_q_reset_immunity_p);
// Property: Verify idle_q only takes valid MuBi4
    values
property idle_q_valid_values_p;
    @(posedge clk_i) disable iff (!rst_ni)
    (1'b1) |-> (idle_q inside {prim_mubi_pkg::
        MuBi4True, prim_mubi_pkg::MuBi4False});
endproperty
IDLE_Q_VALID_VALUES: assert property(
    idle_q_valid_values_p);
```

Listing 3: Generated assertions for uncovered line in Listing 1

V. Results

Table I presents our findings organized into stages. We use JasperGold [12] as the FV tool for our experiments.

Stage 1: In this stage, we generate the verification plan and assertions based on the input specification and the protocol diagrams, as depicted in Figure 1 with our DUT. With the help of the statement coverage model, we first find the uncovered statements with scope restricted to the DUT and its direct inclusions only. The details of our findings for each protocol is shown in the table for Stage 1. For GPIO and RV TIMER, no statements were uncovered in the DUT while HMAC has some statements uncovered.

Stage 2: Using the uncovered lines in the DUT we use our RAG-based framework to explain the uncovered lines, as explained in the previous section. An additional invocation to the LLM is done now with the same inputs as in Stage 1 along with the explanations received on the uncovered lines from RAG. The number of checklists and the corresponding number of assertions are shown beside Stage 2. For GPIO and RV_TIMER, this is not applicable (Stage 1 and Stage 2 are identical) since all lines of the DUT are already covered in Stage 1, thus Stage 2 is not need to be executed. The results for HMAC are given in the row for Stage 2. We also check the coverage metrics as earlier with respect to this new set of assertions. As expected, the coverage for HMAC code improves with this exercise since more informed assertions are generated and facilitated by the explanations.

Stage 3: In this stage, we incorporate the uncovered lines on the files, which are direct inclusions in the DUT code. As in Stage 2, we carry out the RAG to generate explanations for these uncovered lines, with a specific query to explain the coverage gap in the dependency module in the context of the DUT. These explanations serve as additional input to generate a fresh set of checklists and assertions. The results are reported beside the row for Stage 3 for each protocol.

Throughout all three stages, if any syntactical errors are found in the LLM-generated assertions, we fix them by rerunning the setup. In Table I, the column **# Shots** describes the number of iterations needed to obtain a correct set of SVAs. The important point to note is that we make 2 implicit (through RAG for explanations) and 4 explicit LLM invocations (1 each in the plan generation and assertion generation steps of Stage 1, and then 2 in each of Stage 2 and Stage 3), with each invocation costing around 2000 tokens. Notably, the verification plan and assertion generation phases involve reading the full specification document and RTL files, which increases token usage due to the long input size. However, models like Claude 3.5 support extended context windows, allowing us to process complete RTL modules without truncation. As shown in the rightmost column of Table I, the total token count varies across stages and designs: Stage 1 typically consumes 13K–20K tokens, while in Stage 2 and 3, due to the inclusion of RAG and question-answering for uncovered lines, the token usage increases significantly. For GPIO and RV Timer, no uncovered lines were found in Stage 1, so we directly move to Stage 3, resulting in 0 token cost in Stage 2. As the number of RTL lines increases or repetitive error corrections are needed, this token cost becomes a burden. Therefore, we limit our processing to the DUT and its directly included files only.

Table I presents verification results for GPIO.sv, RV_Timer.sv, and HMAC.sv across the three stages. For each IP and stage, the table shows the number of verification checklists (nChklst), generated assertions (nAst), iteration attempts (# Shots), and pass rate percentage (PR). Coverage metrics are provided in two categories: for the DUT alone and the DUT with its dependencies. Each category includes formal coverage (F Cov), stimuli coverage (S Cov), and checker coverage (C Cov) percentages. The *Token Count* shows the total expended tokens during each stage. Across all three modules, several patterns emerge as described below:

- *Number of Assertions*: Moving across stages consistently increases the number of checklists and assertions significantly (GPIO: 13→113 assertions, RV_Timer: 5→75 assertions, HMAC: 7→29→35 assertions).
- *Low PR*: The pass rate decreases from Stage 1 to 3. In Stages 1-2, it is calculated only for the DUT, while in

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on March 14,2026 at 05:07:33 UTC from IEEE Xplore. Restrictions apply.

IP Name	Setup	nChklst	nAst	# Shots	PR	DUT coverage			DUT + Dependencies			Token Count
						F Cov	S Cov	C Cov	F Cov	S Cov	C Cov	
GPIO	Stage 1	15	13	0	100	100	100	100	80.19	97.46	81.6	13348
	Stage 2	15	13	0	100	100	100	100	80.19	97.46	81.6	0
	Stage 3	89	113	1	81.23	100	100	100	83.25	95.5	86.9	53425
RV Timer	Stage 1	5	5	0	100	100	100	100	62.24	94.6	64.11	7556
	Stage 2	5	5	0	100	100	100	100	62.24	94.6	64.11	0
	Stage 3	50	75	1	60.23	100	100	100	69.24	95.08	69.09	32946
HMAC	Stage 1	25	7	1	100	93.59	100	93.6	70.13	92.03	69.63	20026
	Stage 2	35	29	0	100	96.15	100	96.15	82.43	92.03	88.57	20590
	Stage 3	45	35	1	90.56	96.15	100	96.15	84.35	92.56	88.86	18531

TABLE I: Verification results for OpenTitan IP modules at different stages.

Stage 3, it includes all dependencies, thus, expanded scope introduces more complexity, resulting in a lower pass rate despite improved overall coverage. It might be noted that the LLM in any of the stages has no other source file except the DUT source as input. Thus, the assertions generated are based only on DUT local knowledge and the explanations.

- *Consistent Stimuli Coverage*: Though formal and checker coverage fluctuates, stimuli coverage remains high ($> 90\%$ in most cases), suggesting that RAG excels at generating tests that exercise the design space effectively.
- *Direct Improvement in Formal Coverage*: For HMAC.sv, RAG on uncovered lines improves from 93.59% to 96.15%.
- *Maintenance of High Stimuli Coverage*: Even in complex scenarios, RAG maintains high stimuli coverage (mostly $>90\%$), ensuring thorough design space exploration.
- *Scalability to Complex Verification Scenarios*: When moving to Stage 1 with direct dependencies, the number of checklists and assertions increases dramatically, yet coverage metrics remain strong, suggesting RAG provides valuable context for more complex verification tasks.
- *Low Correction Rates*: The number of shots is 0 / 1 across all configurations. This minimal correction refers to the rare need for changes like fixing inconsistent variable names and addressing wrong reset/clock domain assignments in the generated assertions, suggesting that the generated assertions require little to no human intervention.

VI. Conclusion

This work introduces a novel framework that effectively transforms natural language specifications into high-quality SVAs with minimal human intervention. Our contributions include: (1) an efficient SVA generation approach that achieves syntactically correct assertions with zero or one-shot prompting, (2) integration of RAG and CoT for formal verification of complex hardware designs like OpenTitan IPs, and (3) a coverage-driven refinement process that systematically improves verification quality. LISA's ability to generate assertions with minimal retries is useful for hardware verification. As future work, we aim to: (i) evaluate runtime and retrieval latency metrics; (ii) scale LISA to handle complex hierarchical SoC designs; (iii) expand coverage evaluation beyond statement metrics to include FSM state transitions, property coverage, and bug detection; and (iv) explore faster, more accurate RAG indexing strategies for better context retrieval.

References

[1] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *CoRR*, vol. abs/2406.00515, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2406.00515

[2] M. Cosler, C. Hahn *et al.*, "nl2spec: Interactively translating unstructured natural language to temporal logics with large language models," 2023. [Online]. Available: https://arxiv.org/abs/2303.04864

[3] Q. Zhang, W. Sun *et al.*, "Exploring automated assertion generation via large language models," *ACM Trans. Softw. Eng. Methodol.*, 2024.

[4] M. Shahidzadeh, B. Ghavami *et al.*, "Automatic high-quality verilog assertion generation through subtask-focused fine-tuned llms and iterative prompting," *CoRR*, vol. abs/2411.15442, 2024.

[5] M. Hassan, R. Drechsler *et al.*, "Llm-guided formal verification coupled with mutation testing," in *DATE*, 2024, pp. 1–2.

[6] R. Kande, H. Pearce *et al.*, "(security) assertions by large language models," *IEEE Transactions on Information Forensics and Security*, vol. 19, p. 4374–4389, 2024. [Online]. Available: http://dx.doi.org/10.1109/TIFS.2024.3372809

[7] C. Sun *et al.*, "Towards improving verification productivity with circuit-aware translation of natural language to systemverilog assertions," in *First International Workshop on Deep Learning-aided Verification*, 2023.

[8] M. Orenes-Vera, M. Martonosi, and D. Wentzlaff, "Using llms to facilitate formal verification of RTL," *CoRR*, vol. abs/2309.09437, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2309.09437

[9] B. Mali, K. Maddala *et al.*, " ChIRAAG: ChatGPT Informed Rapid and Automated Assertion Generation ," in *ISVLSI*, 2024, pp. 680–683.

[10] lowRISC, "Opentitan." [Online]. Available: https://github.com/lowRISC/opentitan

[11] J. Wei, X. Wang *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," 2023. [Online]. Available: https://arxiv.org/abs/2201.11903

[12] "Jaspergold." [Online]. Available: https://www.cadence.com/en_US/home/tools/system-design-and-verification/formal-and-static-verification/jasper-gold-verification-platform/formal-property-verification-app.html

[13] Anthropic, "Claude ai," 2024. [Online]. Available: https://www.anthropic.com/news/claude-3-5-sonnet

[14] M. Chen, J. Tworek *et al.*, "Evaluating large language models trained on code," 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[15] S. Mandal, A. Chethan *et al.*, "Large language models based automatic synthesis of software specifications," 2023. [Online]. Available: https://arxiv.org/abs/2304.09181

[16] P. P. Anshul Jain, Karan Rawat, "Genai leap in formal verification testplanning," DVCON India 2024.

[17] A. Ayalasomayajula *et al.*, "LASP: LLM assisted security property generation for soc verification," in *MLCAD*, 2024, pp. 29:1–29:7.

[18] S. Thakur, B. Ahmad *et al.*, "Verigen: A large language model for verilog code generation," 2023. [Online]. Available: https://arxiv.org/abs/2308.00708

[19] M. Gao, J. Zhao *et al.*, "Autovcoder: A systematic framework for automated verilog code generation using llms," 2024. [Online]. Available: https://arxiv.org/abs/2407.18333

[20] N. Reimers and I. Gurevych, "Sentence-bert: Sentence embeddings using siamese bert-networks," 2019. [Online]. Available: https://arxiv.org/abs/1908.10084

[21] M. Douze *et al.*, "The faiss library," *CoRR*, vol. abs/2401.08281, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2401.08281

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI. Downloaded on March 14,2026 at 05:07:33 UTC from IEEE Xplore. Restrictions apply.